

Informe de reflexión: trabajo final CI/CD

Sistemas Distribuidos, Parte II

Jordy Romero

Nayeli Barbecho

24 de julio de 2026

Repositorio: <https://github.com/jordyromero03/ExamenFinalDistribuidos>. Contiene el código, los manifiestos, el README con evidencia real de cada etapa y la bitácora completa de despliegues en docs/deploy-log.md.

1 Justificación de la estrategia de despliegue elegida: Blue-Green

Se descartó Canary a favor de Blue-Green por el perfil concreto de inventario-app: es una aplicación de bajo tráfico, sin usuarios reales en producción (un ejercicio de laboratorio), donde el objetivo pedagógico es demostrar un mecanismo de corte de tráfico controlado y verificable, no optimizar el riesgo de exponer una fracción de usuarios reales a una versión sin probar, que es la ventaja real que justificaría un Canary. Blue-Green, en cambio, se apoya puramente en recursos nativos de Kubernetes (dos Deployment y un Service cuyo selector se cambia), y su corte es instantáneo y binario: se puede demostrar con un solo `curl` antes y después del `kubectl patch`, sin necesitar decenas de peticiones repetidas para confirmar estadísticamente un reparto proporcional como exige Canary. Para el caso de esta app, esa simplicidad de verificación pesó más que la ventaja de exposición gradual que ofrece Canary.

2 Observación: persistencia de datos

inventario-app guarda su catálogo en un archivo JSON local (`data/products.json`), sin ningún `PersistentVolume` montado. Con 2 réplicas del Deployment base, cada pod tiene su propia copia aislada de ese archivo: un producto creado por una petición puede no aparecer en la siguiente lectura si el Service la enruta a otro pod, sin que se haya perdido nada; simplemente el otro pod nunca se enteró de que existía. Al recrear un pod (`kubectl delete pod`), el reemplazo arranca desde cero con los 3 productos semilla del código, porque la capa de escritura donde vivía el dato nuevo se destruye junto con el contenedor viejo. No es un error de la aplicación, sino la consecuencia esperada de usar disco local en un entorno donde los pods son efímeros por diseño. En un sistema real, ese estado se externalizaría a una base de datos con su propio volumen o a un servicio gestionado fuera del clúster.

3 Problemas reales encontrados y cómo se resolvieron

- **Tag de acción de terceros mal escrito.** `aquasecurity/trivy-action@0.24.0` hizo fallar el pipeline dos veces seguidas (`Unable to resolve action`), porque los tags de esa acción llevan el prefijo `v` (`v0.24.0`). Se corrigió consultando los tags reales con `gh api`.
- **CVE real en una dependencia transitiva.** Al bajar `express` a `4.17.1` a propósito para probar el escaneo de Trivy, este encontró `CVE-2026-59873 (CRITICAL)` en `tar`, una dependencia transitiva de `express` y no del propio paquete. Esto confirma que un escaneo de imagen detecta riesgos que revisar solo el `package.json` directo no vería.
- **CVE en la imagen base, no en la app.** Tras revertir `express`, el mismo CVE de `tar` seguía apareciendo. Se investigó hasta encontrar que venía empaquetado dentro del propio `npm` que trae `node:20-alpine`, y ni siquiera una imagen base más nueva (`node:22-alpine`) lo tenía parchado todavía. Se resolvió eliminando `npm` de la etapa final del `Dockerfile`, ya que la aplicación no lo necesita en tiempo de ejecución.
- **Selector de Service compartido entre Deployments.** El Service base y los pods de Blue-Green compartían la label `app: inventario-app`, así que el Service base terminaba repartiendo tráfico también hacia pods de Blue-Green. Se resolvió agregando una label distintiva (`slot: stable`) al Deployment base y a su Service.
- **Arranque lento sin aplicar de verdad.** Al agregar `STARTUP_DELAY_SECONDS`, dos intentos fallaron por

separado: primero faltaba la variable de entorno en el manifiesto, y después el código de `server.js` nunca se había `commit`ado ni `push`ado, por lo que la imagen en el clúster seguía siendo la anterior. Ambos casos se corrigieron y se confirmaron con la transición real de 0/1 a 1/1 tomando los ~15 segundos esperados.

4 Métricas DORA

Calculadas a partir de `docs/deploy-log.md`, con timestamps reales de commits, runs de GitHub Actions y `ReplicaSet` creados en el clúster, no con estimaciones.

Métrica	Valor real	Cómo se calculó
Lead time for changes	2 min 14 s / 13 min 54 s	Dos cambios de código: tiempo entre el commit y el nuevo <code>ReplicaSet</code> confirmado en el clúster.
Frecuencia de despliegue	≈ 5/día	10 promociones reales al clúster en 2 días de trabajo.
Change failure rate	40%	4 de 10 runs de pipeline fallaron y necesitaron un commit de corrección posterior.

Ubicación en la tabla de niveles DORA (reportes Accelerate State of DevOps, Google/DORA)

Nivel	Deployment frequency	Lead time	Change failure rate
Elite	Bajo demanda (varias/día)	Menos de 1 hora	0–15%
Alto	1/semana a 1/mes	1 día a 1 semana	16–30%
Medio	1/mes a 1/6 meses	1 mes a 6 meses	16–30%
Bajo	Menos de 1/6 meses	Más de 6 meses	16–30%

Por frecuencia de despliegue (~5/día) y lead time (en minutos), el proyecto se ubicaría en el nivel **Elite**. El **change failure rate de 40% cae en el nivel más bajo** de la tabla. Esta combinación no es contradictoria: DORA mide equipos maduros operando en producción real durante meses, mientras que este trabajo lo hizo una sola persona, en 2 días, aprendiendo Kubernetes, Docker y GitHub Actions por primera vez, con la consigna pidiendo explícitamente provocar fallos a propósito (en Trivy y en readiness) para demostrar que el pipeline los detecta. Cada fallo real tuvo una causa distinta y una corrección concreta documentada. De hecho, la métrica que más se acerca a reflejar experiencia real, el change failure rate, es también la que más rápido mejoraría con historia: descontando los 2 fallos repetidos por el mismo bug del tag de Trivy (que en un intento único habría sido 1 fallo, no 2), el failure rate de causas distintas baja a 3/9, aproximadamente 33%, todavía bajo pero visiblemente más cerca del nivel Medio. La velocidad, en cambio, sí refleja algo genuino: con el principio de build once, promote many y un pipeline de fail fast bien armado, promover un cambio real al clúster tomó minutos, no días.